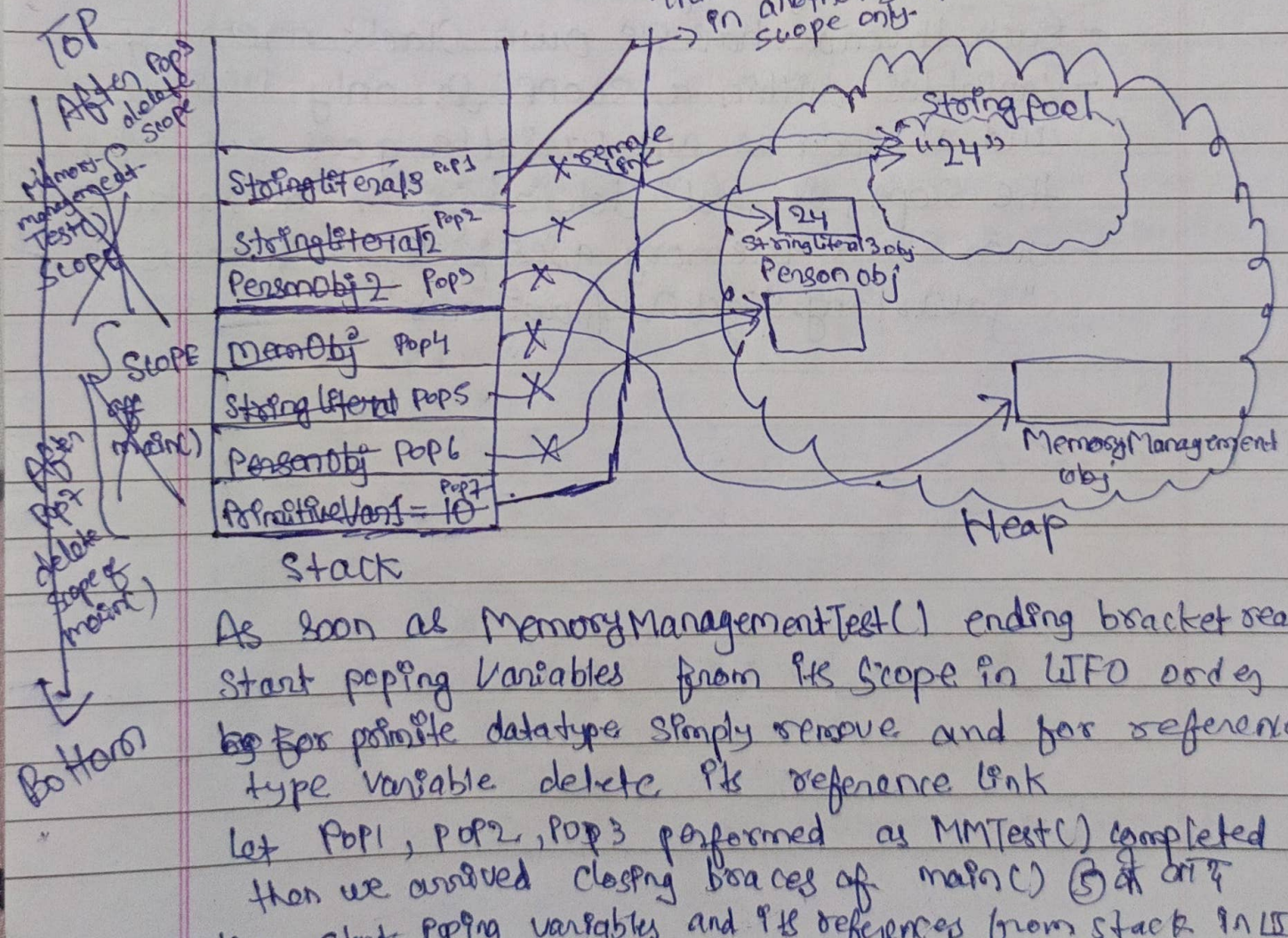
Java Memory Mgmt and Garbage Collection

2 Types of Memory
- STACK
- HEAP

Both stack & heap are created by JVM and Stored in RAM

- STACK Memory:
  - Store temporary variables and seperate memory block for methods
  - Store primitive data types
  - Store Reference of the heap objects
    - Strong reference
    - Weak reference
      - Soft reference
  - Each thread has its own Stack memory.
  - Variables within a SCOPE is only visible and as soon as any variable goes out of the scope, it get deleted from the stack (in LIFO order)
  - When Stack memory goes full, it throws "java.lang.StackOverflowError"

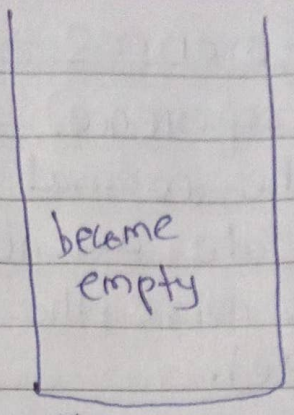
```
Public class MemoryManagement {
    psvm() {
①       int primitiveVariable1 = 10;
②       Person personObj = new Person();
③       String StringLiteral = "24";
④       MemoryManagement memObj = new MemoryManagement();
⑤       memObj.memoryManagementTest(personObj);
    }

    private void memoryManagementTest(Person personObj) {
⑥       Person personObj2 = personObj;
⑦       String stringLiteral2 = "24";
⑧       String stringLiteral3 = new String("24");
    }
}
```



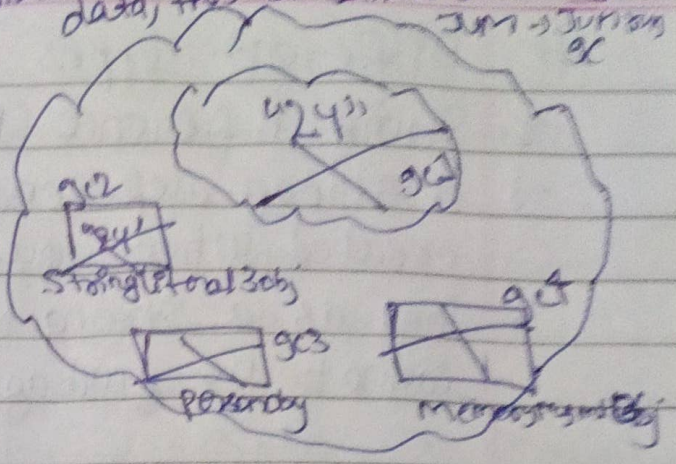
As soon as MemoryManagementTest() ending bracket reached start poping variables from its scope in LIFO order for primitive datatype simply remove and for reference type variable delete its reference link

Let Pop1, Pop2, Pop3 performed as MMTest() completed then we arrived closing braces of main() ⑤ at an? then start poping variables and its references from stack in LIFO

We have noticed something Stack is empty but heap still contain data, this all will be done by JVM → Jurian sir



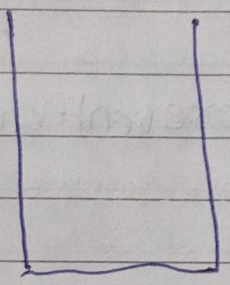
Stack

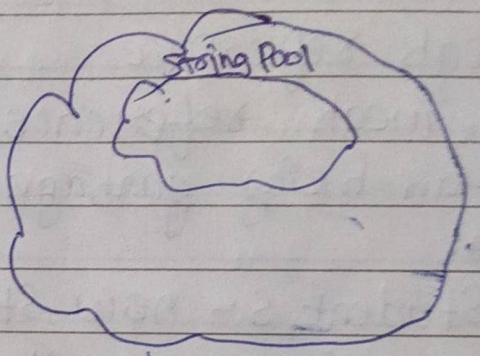


Heap

But heap contain the values only its reference become null

Now Garbage collector come in picture whose task is to remove/clean heap whose reference is "null"





Heap

After GC cycle complete our heap and stack looks like above fully clean & clear.

System.gc(); // calls to explicity run garbage collection process but it totally depends on JVM Sometime it works and sometime it not.

conclusion : Garbage collection process is fully under the control of JVM and its execute gc() periodically. If memory space is going to full then gc() run too freequently, so as a programmer we don't have to think about gc() JVM handle it. MM not "Automatic Memory Mangement happen in JAVA".

Different types of References → 4 types

1. Strong Reference (Default reference)

→ A strong reference is the normal reference created when you instantiate an object. As long as a strong reference exists, the object cannot be garbage collected.

```
Student S = new Student();
```

Garbage collection:

X object is not eligible for garbage collection while a strong reference exists.

2. Weak reference

→ A weak reference does not prevent an object from being garbage collected.

Eg:

```
Student S = new Student();
WeakReference<Student> weakRef =
        new WeakReference<>(S);
S = null;
```

- Garbage collection:
  ✓ object can be collected during the next GC cycle, even if memory is sufficient.
- Use case
  → WeakHashMap
  → Caching where objects should disappear if no longer strongly referenced.

3. Soft Reference
→ A soft reference keeps an object alive until the JVM needs memory
→ SoftReference<Student> softRef = new SoftReference<>(new Student());

Garbage collection:
✓ collected only when the JVM is low on Memory or memory scarsity.

Use cases:
- Image caches
- Memory-Sensitive caches.

4. Phantom Reference (Advanced)
→ A phantom reference is the weakest type of reference. It is used to perform cleanup after an object has been finalized and is about to be reclaimed.

Eg:
ReferenceQueue<Student> queue = new ReferenceQueue<>();
PhantomReference<Student> phantom = new PhantomReference<>(new Student(), queue);

Garbage Collection:
- Cannot retrieve the object using get() (it always returns null)
- used with ReferenceQueue for advance resource cleanup

Use Cases:
- Managing off-heap/native resources
- Advance memory mgmt